Chapter 3

Stacks

Stacks

- A *stack* is a linear list in which all additions and deletions are restricted to one end, called the *top*.
- Last in – first out (LIFO) data structure: The last item put on the stack is the first item that can be taken off.

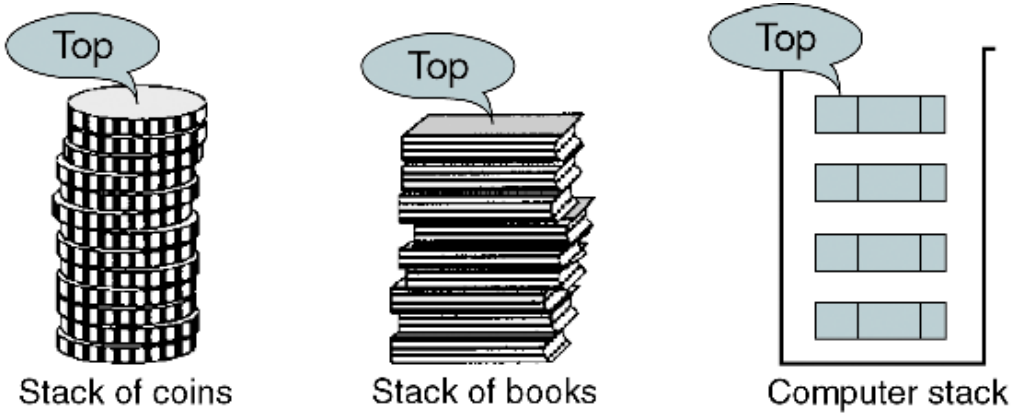


FIGURE 3-1 Stack

3-1 Basic Stack Operations

- Push
- Pop
- Stack Top

Push

- Push adds an item at the top of the stack. After the push, the new item becomes the top.
- We must ensure that there is room for the new item. If there is not enough room, the stack is in an overflow state and the item cannot be added

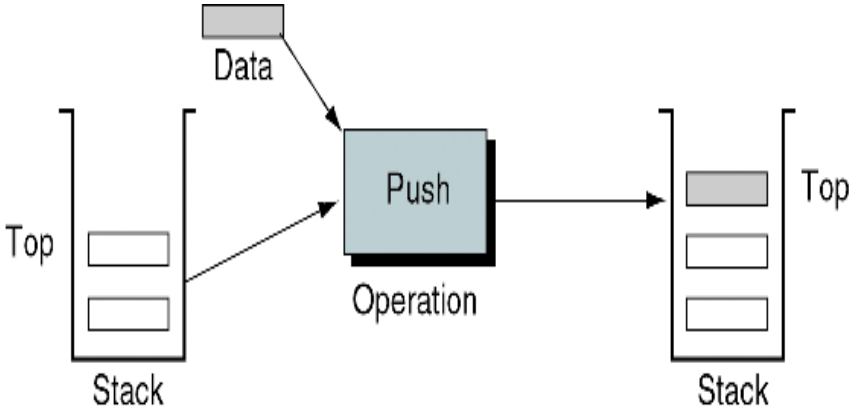


FIGURE 3-2 Push Stack Operation

Pop

- Remove the item at the top of the stack and return it to the user.
- The next older item in the stack becomes the top.
- When the last item in the stack is deleted, the stack must be set to its empty state.
- If pop is called when the stack is empty, it is in an underflow state

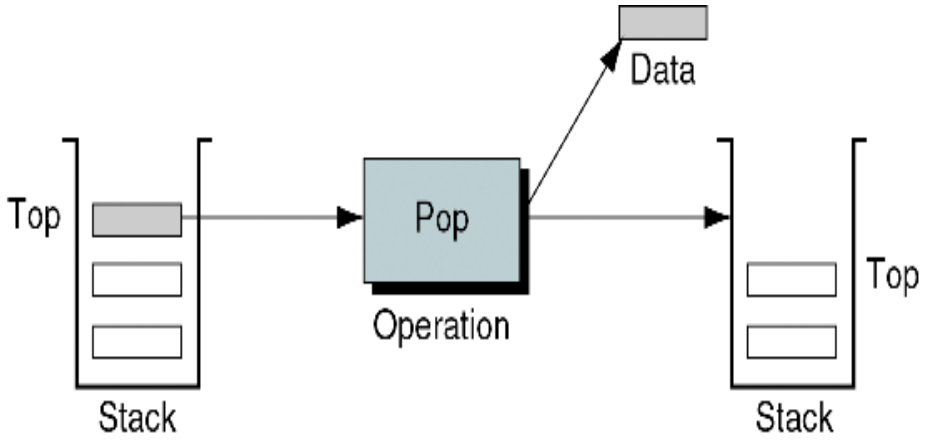


FIGURE 3-3 Pop Stack Operation

Stack top

- Stack top copies the item at the top of the stack but does not delete it.
- You might think of this operation as reading the stack top.
- Stack top can also result in underflow if the stack is empty

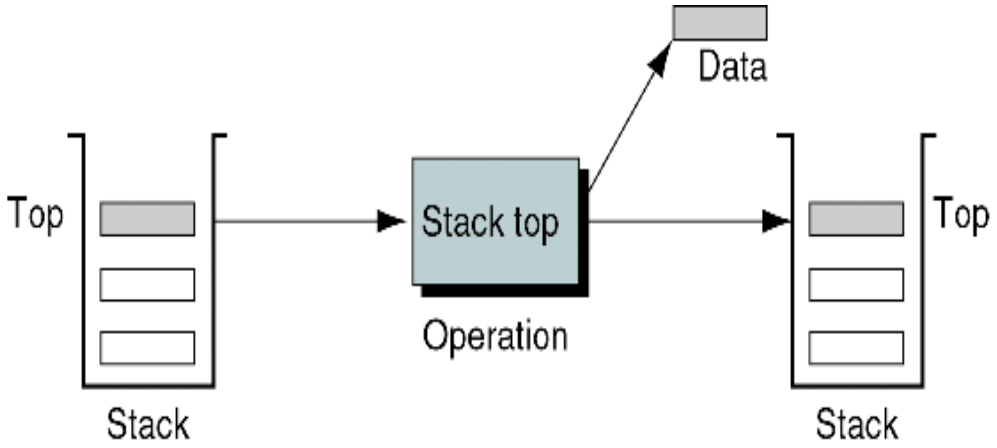


FIGURE 3-4 Stack Top Operation

Example

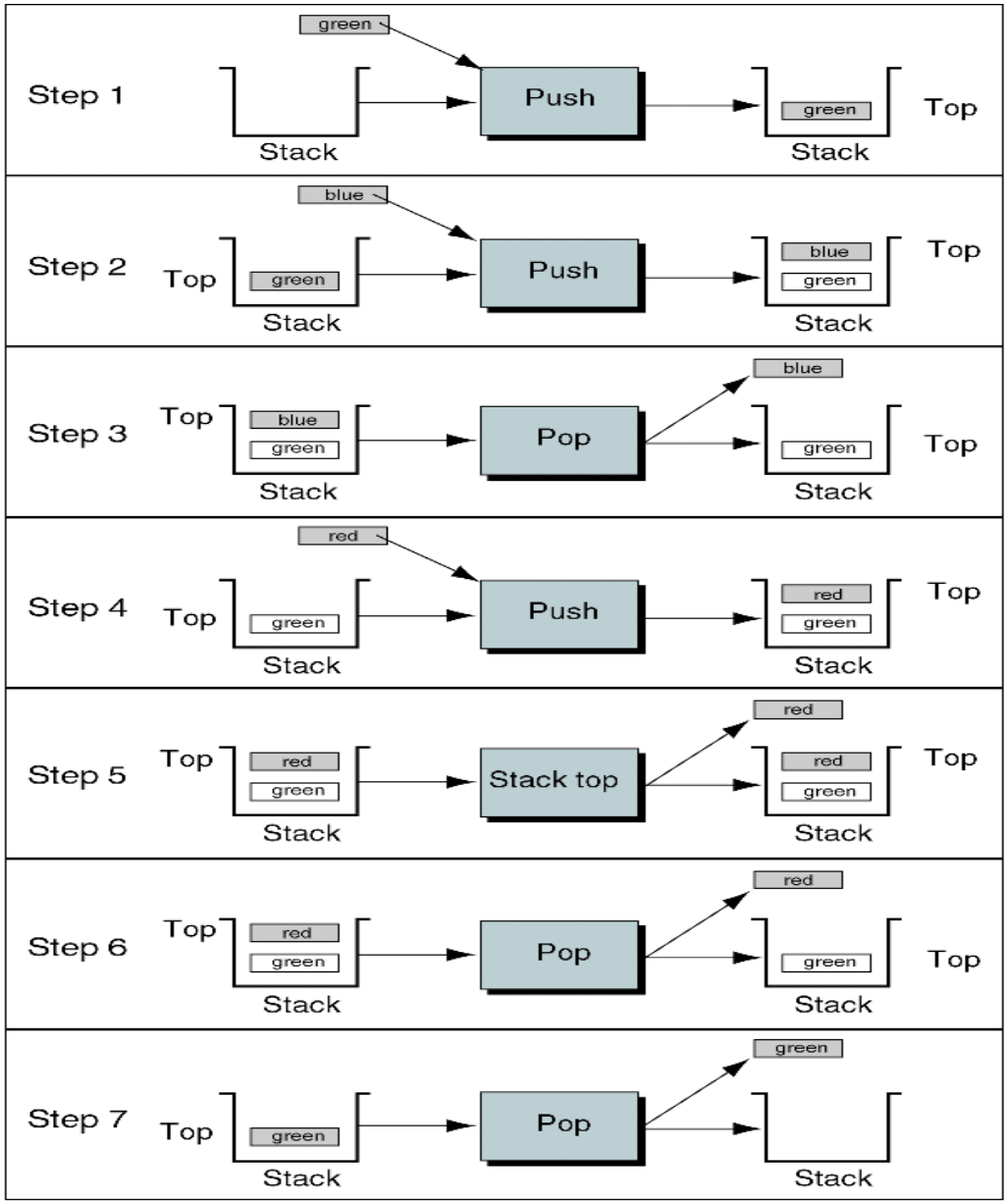


FIGURE 3-5 Stack Example

3-2 Stack Linked List Implementation

Data Structure

- implement the stack as a linked list

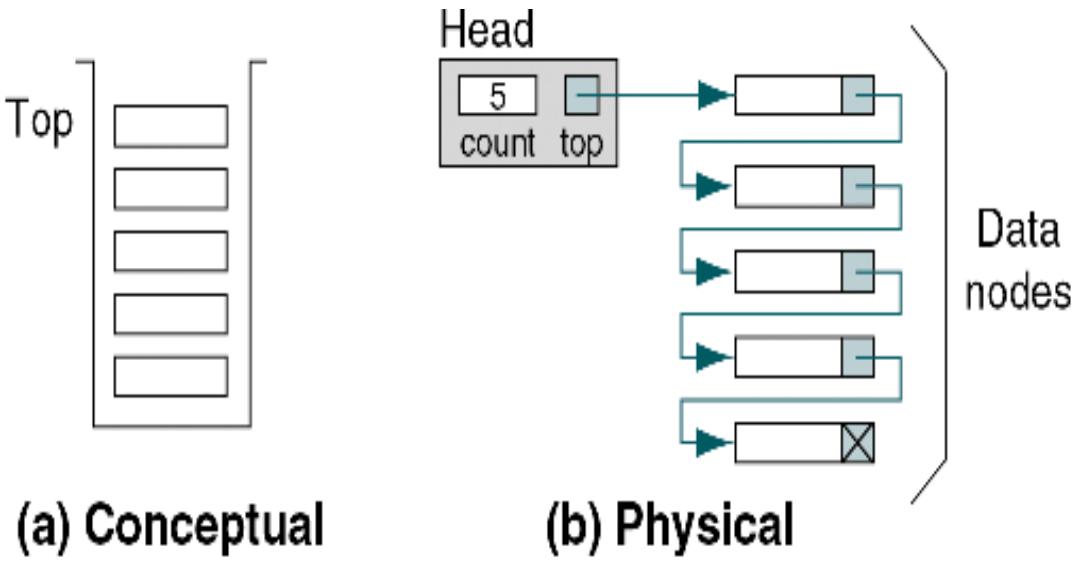
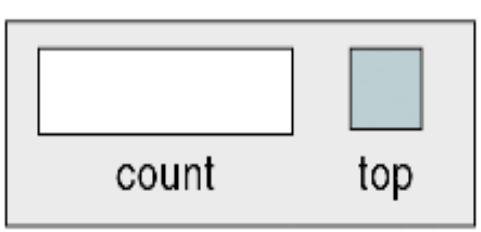
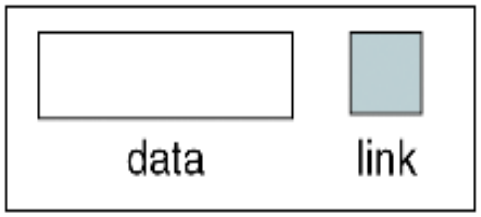


FIGURE 3-6 Conceptual and Physical Stack Implementations



Stack head structure



Stack node structure

```
stack
   count
   top
end stack


node
   data
   link
end node
```

FIGURE 3-7 Stack Data Structure

Stack Algorithms

- Create stack
- Push stack
- Pop stack
- Stack top
- Empty stack
- Full stack
- Stack count
- Destroy stack

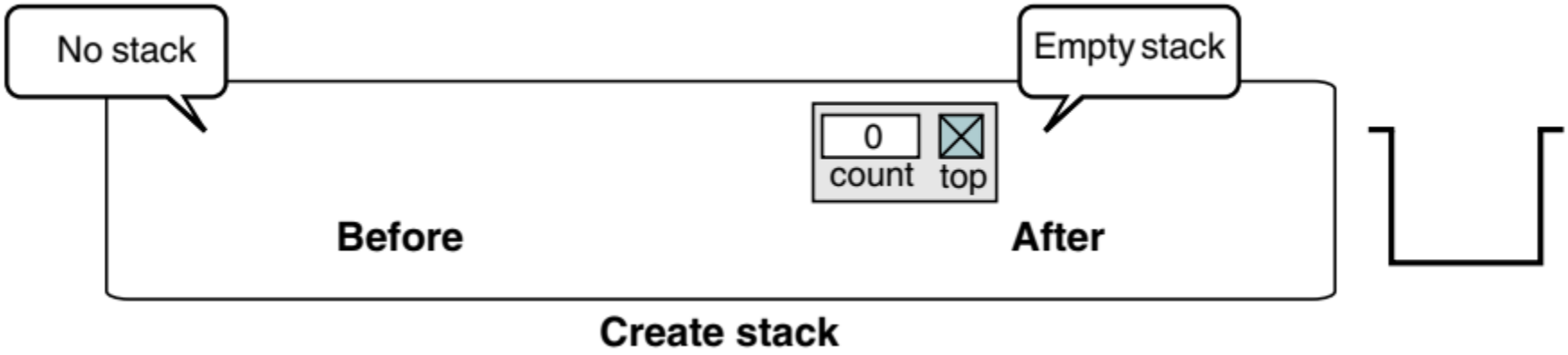
Create stack

ALGORITHM 3-1 Create Stack

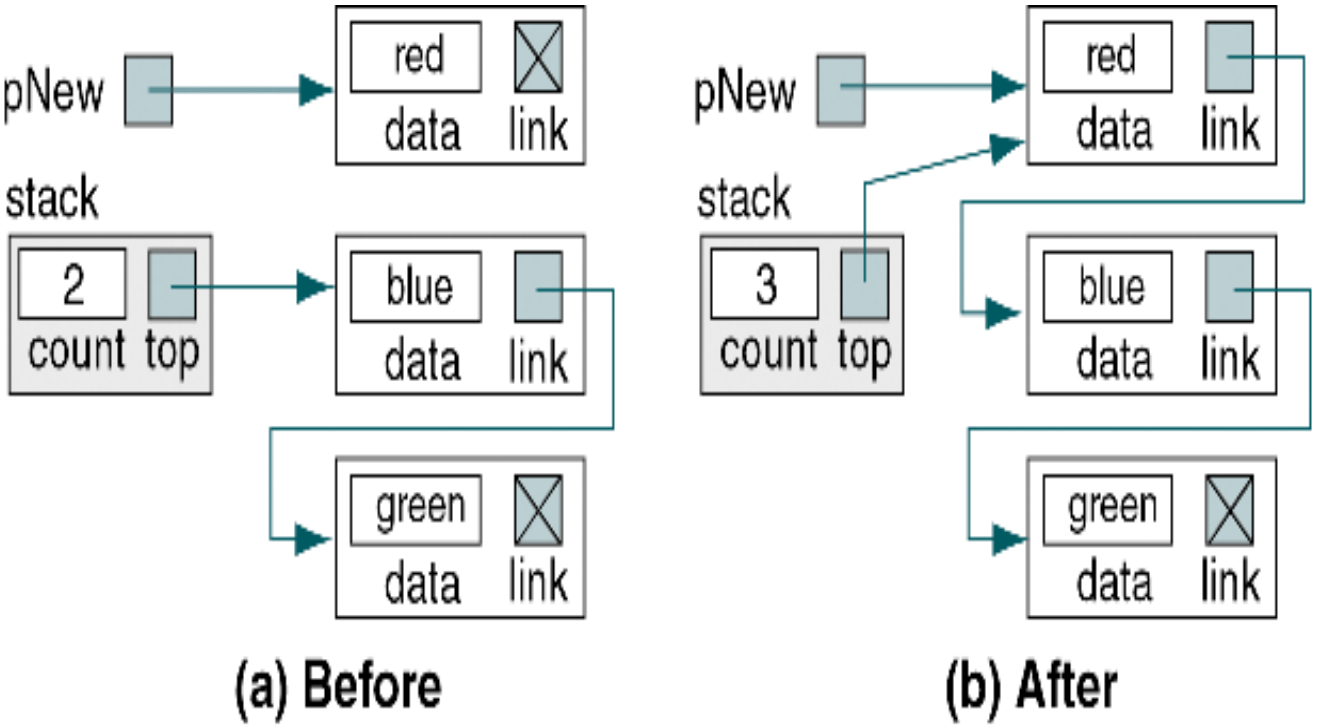
```
Algorithm createStack
Creates and initializes metadata structure.
   Pre    Nothing
   Post   Structure created and initialized
   Return stack head
1  allocate memory for stack head
2  set count to 0
3  set top to null
4  return stack head
end createStack
```

Create stack allocates memory for the stack structure and initializes its metadata.

Create stack



Push stack example



- Point the new node to the top node
- Point top to the new node.

Push stack algorithm

ALGORITHM 3-2 Push Stack Design

```
Algorithm pushStack (stack, data)
Insert (push) one item into the stack.
   Pre  stack passed by reference
        data contain data to be pushed into stack
   Post data have been pushed in stack
 1 allocate new node
 2 store data in new node
 3 make current top node the second node
 4 make new node the top
 5 increment stack count
end pushStack
```

Pop stack

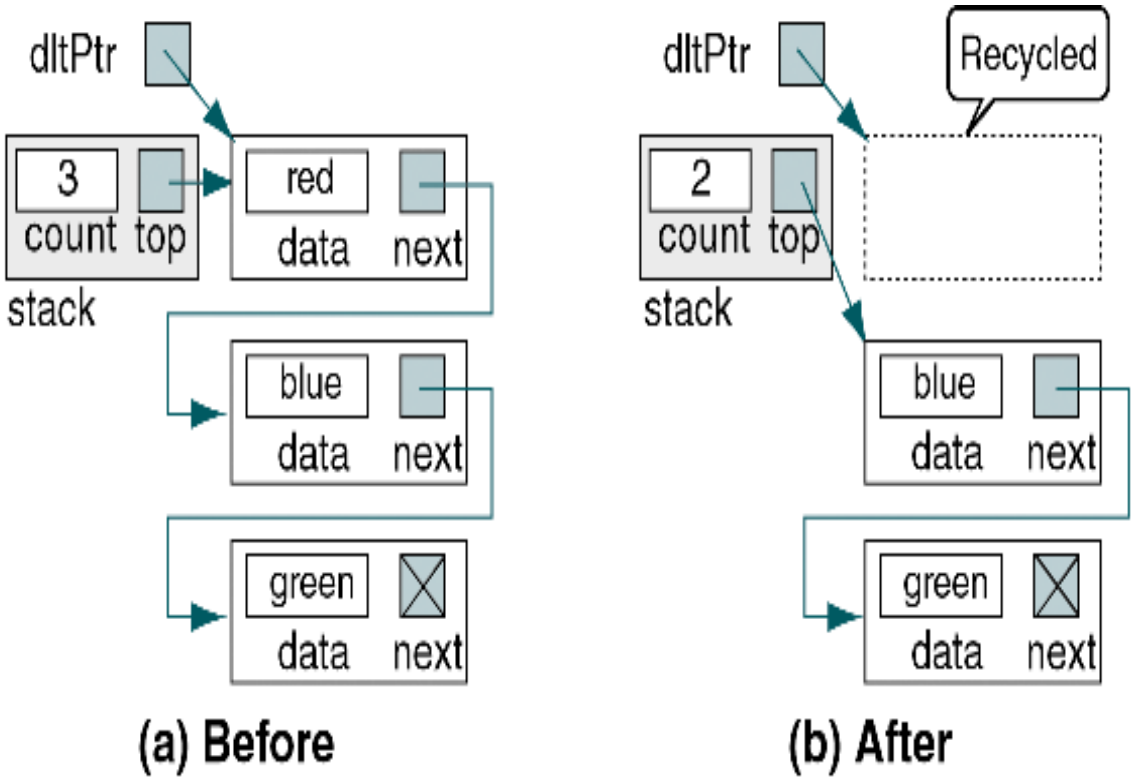


FIGURE 3-10 Pop Stack Example

ALGORITHM 3-3 Pop Stack

```
Algorithm popStack (stack, dataOut)
This algorithm pops the item on the top of the stack and
returns it to the user.
   Pre    stack passed by reference
          dataOut is reference variable to receive data
   Post   Data have been returned to calling algorithm
   Return true if successful; false if underflow
1  if (stack empty)
   1   set success to false
2  else
   1   set dataOut to data in top node
   2   make second node the top node
   3   decrement stack count
   4   set success to true
3  end if
4  return success
end popStack
```

Stack top

ALGORITHM 3-4 Stack Top Pseudocode

```
Algorithm stackTop  (stack, dataOut)
This algorithm retrieves the data from the top of the stack
without changing the stack.
   Pre    stack is metadata structure to a valid stack
          dataOut is reference variable to receive data
   Post   Data have been returned to calling algorithm
   Return true if data returned, false if underflow
1  if (stack empty)
   1   set success to false
2  else
   1   set dataOut to data in top node
   2   set success to true
3  end if
4  return success
end stackTop
```

Empty stack

ALGORITHM 3-5 Empty Stack

```
Algorithm emptyStack (stack)
Determines if stack is empty and returns a Boolean.
   Pre    stack is metadata structure to a valid stack
   Post   returns stack status
   Return true if stack empty, false if stack contains data
1  if (stack count is 0)
   1   return true
2  else
   1   return false
3  end if
end emptyStack
```

Empty stack

ALGORITHM 3-6 Full Stack

```
Algorithm fullStack  (stack)
Determines if stack is full and returns a Boolean.
   Pre    stack is metadata structure to a valid stack
   Post   returns stack status
   Return true if stack full, false if memory available
1  if (memory not available)
   1   return true
2  else
   1   return false
3  end if
end fullStack
```

Stack count

ALGORITHM 3-7 Stack Count

```
Algorithm stackCount (stack)
Returns the number of elements currently in stack.
   Pre    stack is metadata structure to a valid stack
   Post   returns stack count
   Return integer count of number of elements in stack
 1 return (stack count)
end stackCount
```

Destroy stack

ALGORITHM 3-8 Destroy Stack

```
Algorithm destroyStack (stack)
This algorithm releases all nodes back to the dynamic memory.
   Pre    stack passed by reference
   Post   stack empty and all nodes deleted
1  if (stack not empty)
   1   loop (stack not empty)
       1   delete top node
   2   end loop
2  end if
3  delete stack head
end destroyStack
```

Stack Applications

- Reversing data items
- Parsing
- Postponement of processing data items
- Backtracking

Reversing data

- {1 2 3 4} becomes {4 3 2 1}

- Examples for reversing data
  - Reverse a list
  - Convert decimal to binary

Reverse a number series

- Create a stack
- Read a series of numbers and pushes them into the stack.
- Pop the stack and prints the numbers in reverse order

Convert decimal to binary

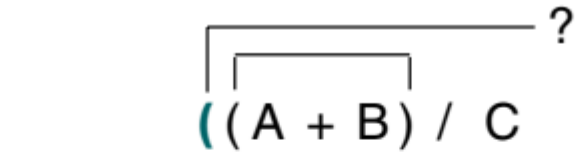
```
1   read (number)
2   loop (number > 0)
    1   set digit to number modulo 2
    2   print (digit)
    3   set number to quotient of number / 2
3   end loop
```

This code has a problem: It creates the binary number backward. (19 becomes 11001 rather than 10011)

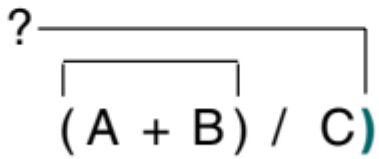
- Instead of printing the binary digit as soon as it is produced, push it into the stack.
- After the number has been completely converted, pop the stack and print the results one digit at a time in a line.

Parsing

- Parsing is any logic that breaks data into independent pieces for further processing.
- Example for parsing
  - Unmatched parentheses



(a) Opening parenthesis not matched



(b) Closing parenthesis not matched

Parse Parentheses

```
Algorithm parseParens
This algorithm reads a source program and parses it to make
sure all opening-closing parentheses are paired.
 1  loop (more data)
    1   read (character)
    2   if (opening parenthesis)
        1   pushStack (stack, character)
    3   else
        1   if (closing parenthesis)
            1   if (emptyStack (stack))
                1   print (Error: Closing parenthesis not matched)
            2   else
                1   popStack(stack)
            3   end if
        2   end if
    4   end if
 2  end loop
 3  if (not emptyStack (stack))
    1   print (Error: Opening parenthesis not matched)
end parseParens
```

Postponement

- Examples for postponement
  - Infix to postfix transformation
  - Postfix expression evaluation

- In an **infix notation**, the operator comes between the two operands, the basic format of the algebraic notation we learned
- In **postfix notation**, the operator comes after its two operands

```
Infix:    A + B
Postfix:  A B +
```

Infix to postfix transformation

- In the infix notation, we need to use parentheses to control the evaluation of the operators.
- We thus have an evaluation method that includes parentheses and two operator priority classes.
  - Priority 0 : (
  - Priority 1 : + -
  - Priority 2 : * /

Manual transformation

1. Fully parenthesize the expression using any explicit parentheses and the arithmetic precedence.
2. Change all infix notations in each parenthesis to postfix notation, starting from the innermost expressions. Conversion to postfix notation is done by moving the operator to the location of the expression's closing parenthesis.
3. Remove all parentheses.

Example

```
A + B * C
```

Step 1 results in

```
( A + ( B * C ) )
```

Step 2 moves the multiply operator after C

```
( A + ( B C * ) )
```

and then moves the addition operator to between the last two closing parentheses. This change is made because the closing parenthesis for the plus sign is the last parenthesis. We now have

```
( A (B C * ) +)
```

Finally, step 3 removes the parentheses.

```
A B C * +
```

More complex example

```
( A + B ) * C  + D + E * F - G
```

Step 1 adds parentheses.

```
( ( ( ( ( A + B ) * C ) + D ) + ( E * F ) ) - G )
```

Step 2 then moves the operators.

```
( ( ( ( ( A B + ) C * ) D + ) ( E F * ) + ) G - )
```

Step 3 removes the parentheses.

```
A B + C * D + E F * + G -
```

Algorithmic transformation

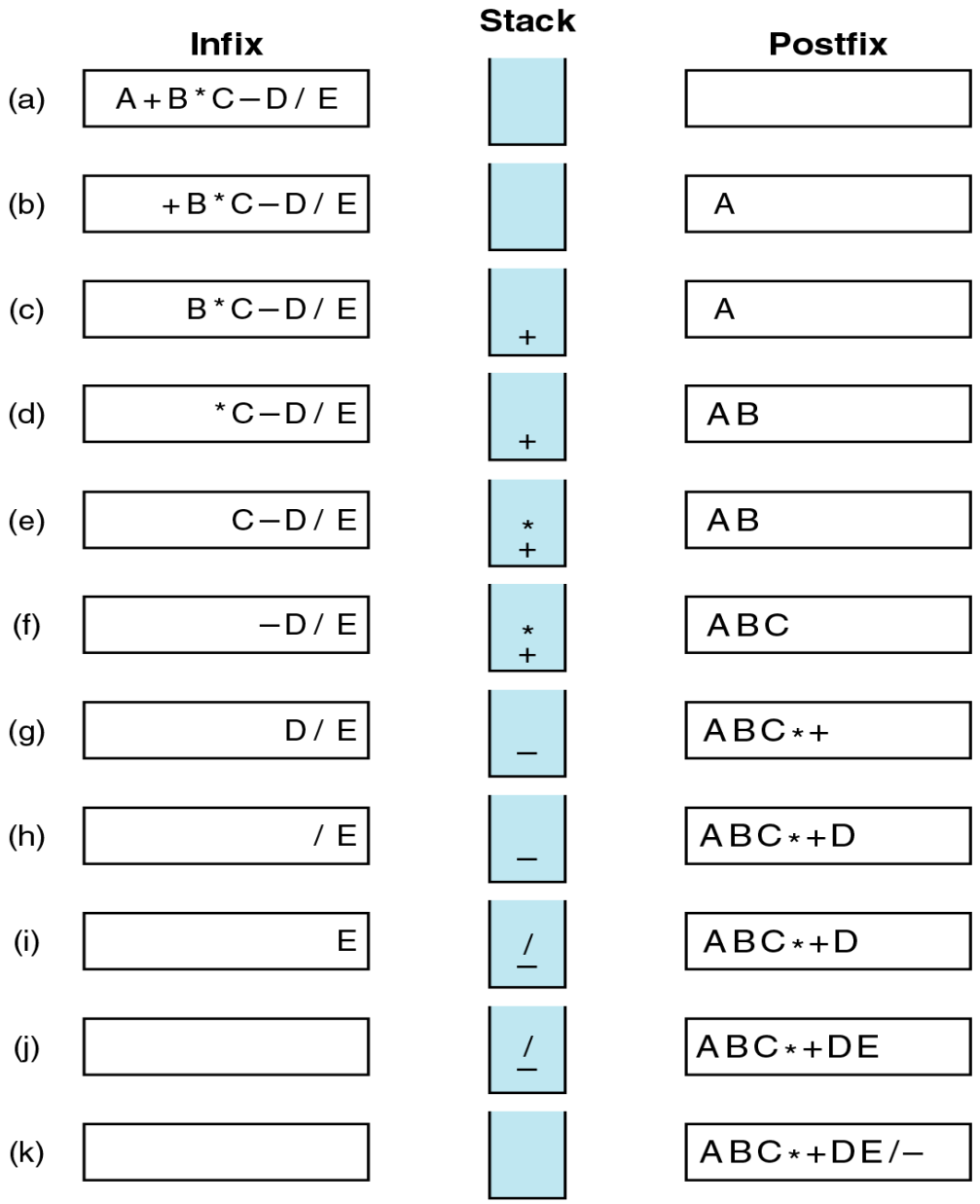
- Copy each operand to output expression
- Push each operator into stack
  - If its priority is higher than the operator at the top of the stack, we go ahead and push it into the stack.
  - If the current operator's priority is lower than or equal to the priorities of all the operators at the top of the stack: pop all these operators and placed in the output expression. Then push the current operator into stack
- While stack is not empty: pop operator and copy to output expression

Infix Transformations

- A+B*C-D/E

  converts to

  ABC*+DE/-



ALGORITHM 3-10 Convert Infix to Postfix

```
Algorithm inToPostFix (formula)
Convert infix formula to postfix.
   Pre    formula is infix notation that has been edited
          to ensure that there are no syntactical errors
   Post   postfix formula has been formatted as a string
   Return postfix formula
1  createStack (stack)
2  loop (for each character in formula)
   1   if (character is open parenthesis)
       1  pushStack (stack, character)
   2   elseif (character is close parenthesis)
       1  popStack (stack, character)
       2  loop (character not open parenthesis)
          1  concatenate character to postFixExpr
          2  popStack (stack, character)
       3  end loop
   3   elseif (character is operator)
          Test priority of token to token at top of stack
       1  stackTop (stack, topToken)
       2  loop (not emptyStack (stack)
            AND priority(character) <= priority(topToken))
          1  popStack (stack, tokenOut)
          2  concatenate tokenOut to postFixExpr
          3  stackTop (stack, topToken)
```

continued

ALGORITHM 3-10 Convert Infix to Postfix *(continued)*

```
      3  end loop
      4  pushStack (stack, token)
   4  else
         Character is operand
      1 Concatenate token to postFixExpr
   5  end if
3  end loop
   Input formula empty. Pop stack to postFix
4  loop (not emptyStack (stack))
   1  popStack (stack, character)
   2  concatenate token to postFixExpr
5  end loop
6  return postFix
end inToPostFix
```

Evaluating postfix expressions

- With each operand in the postfix expression: push it into the stack.
- When we find an operator: pop the two operands at the top of the stack and perform the operator.
- We then push the value back into the stack to be used later.

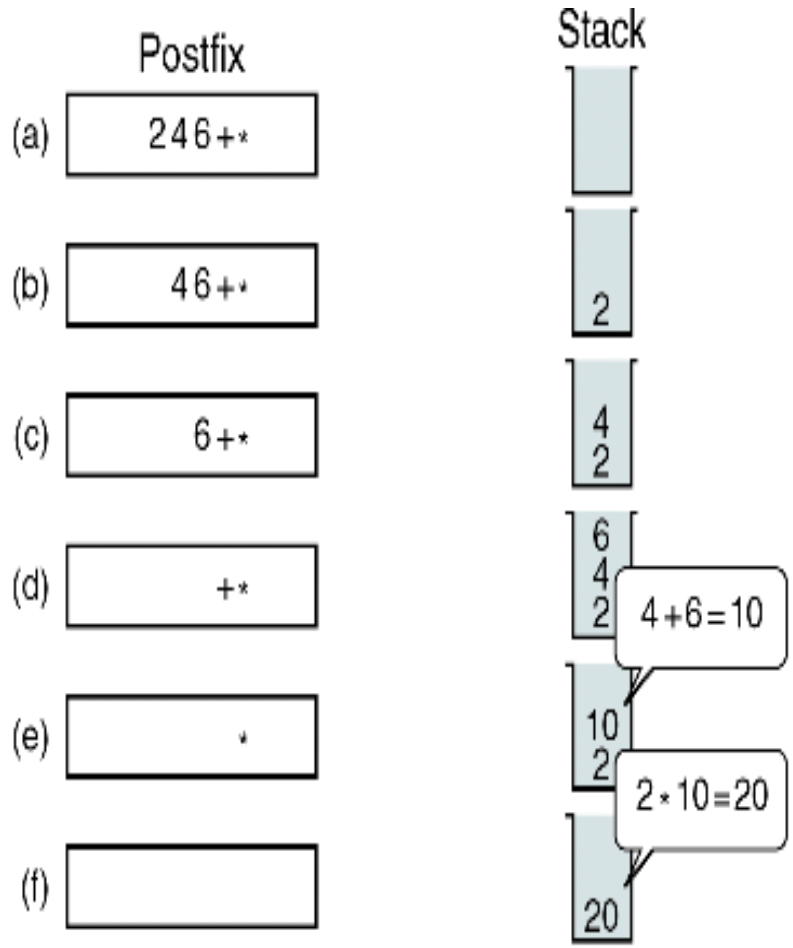


FIGURE 3-16 Evaluation of Postfix Expression

ALGORITHM 3-11 Evaluation of Postfix Expressions

```
Algorithm postFixEvaluate (expr)
This algorithm evaluates a postfix expression and returns its
value.
   Pre    a valid expression
   Post   postfix value computed
   Return value of expression
1  createStack (stack)
2  loop (for each character)
   1   if (character is operand)
       1   pushStack (stack, character)
   2   else
       1   popStack (stack, oper2)
       2   popStack (stack, oper1)
       3   operator = character
       4   set value to calculate (oper1, operator, oper2)
       5   pushStack (stack, value)
   3   end if
3  end loop
4  popStack (stack, result)
5  return (result)
end postFixEvaluate
```

backtracking

- Examples
  - Goal seeking
  - Eight queens problem

Goal seeking

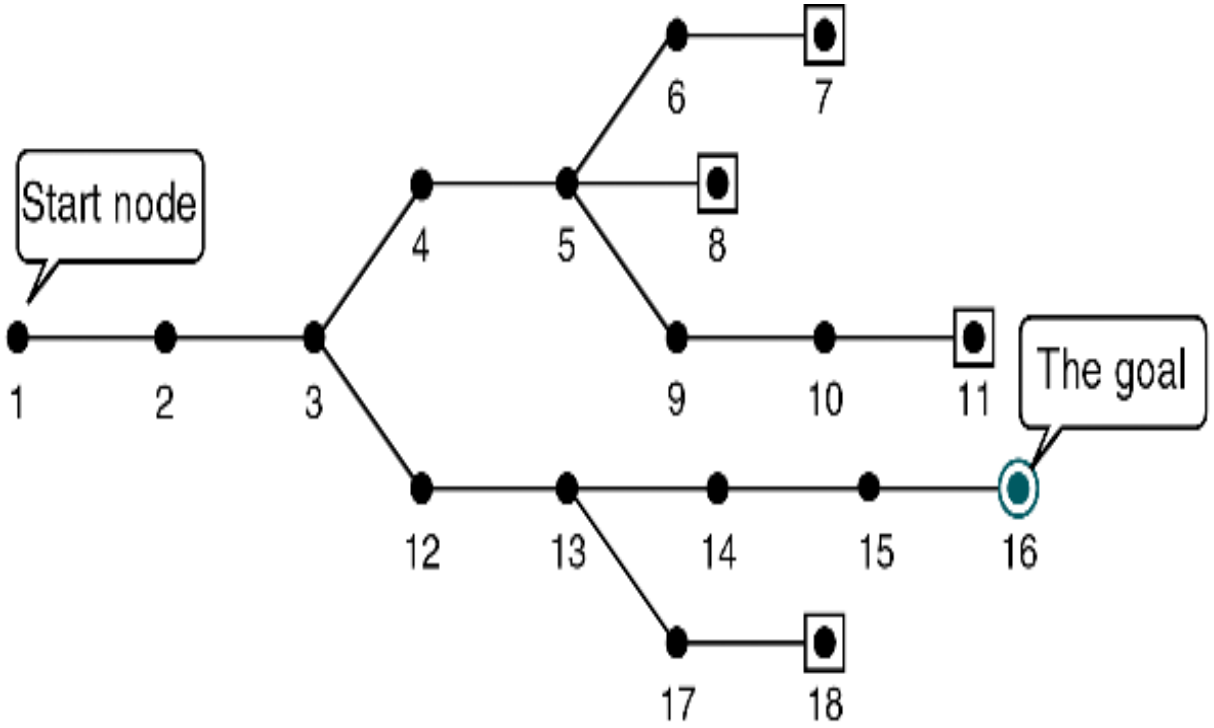
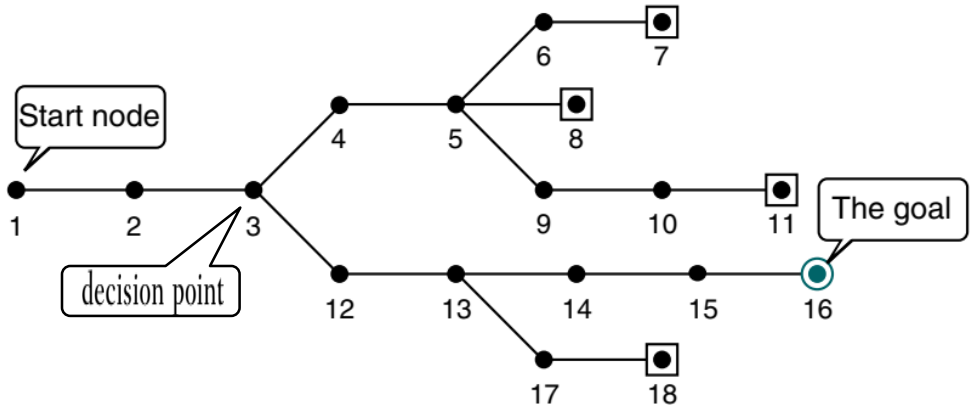
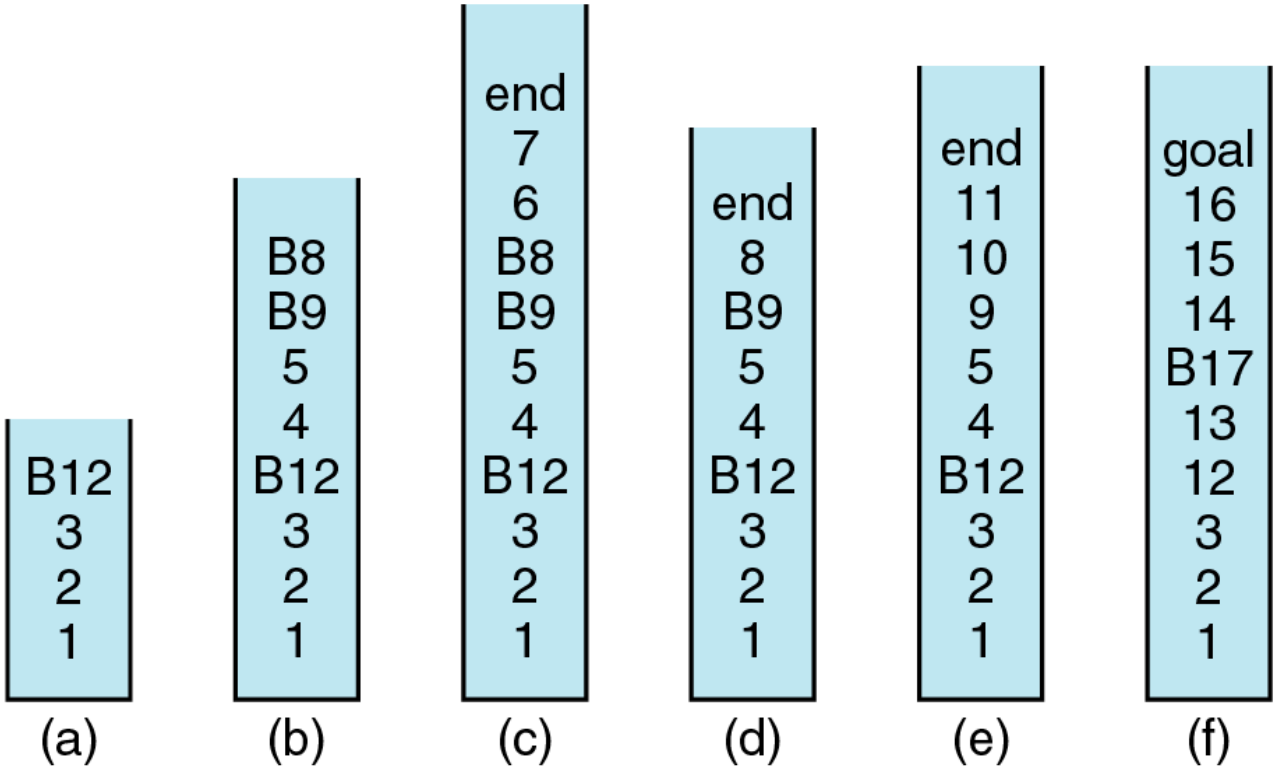


FIGURE 3-17 Backtracking Example

Goal seeking



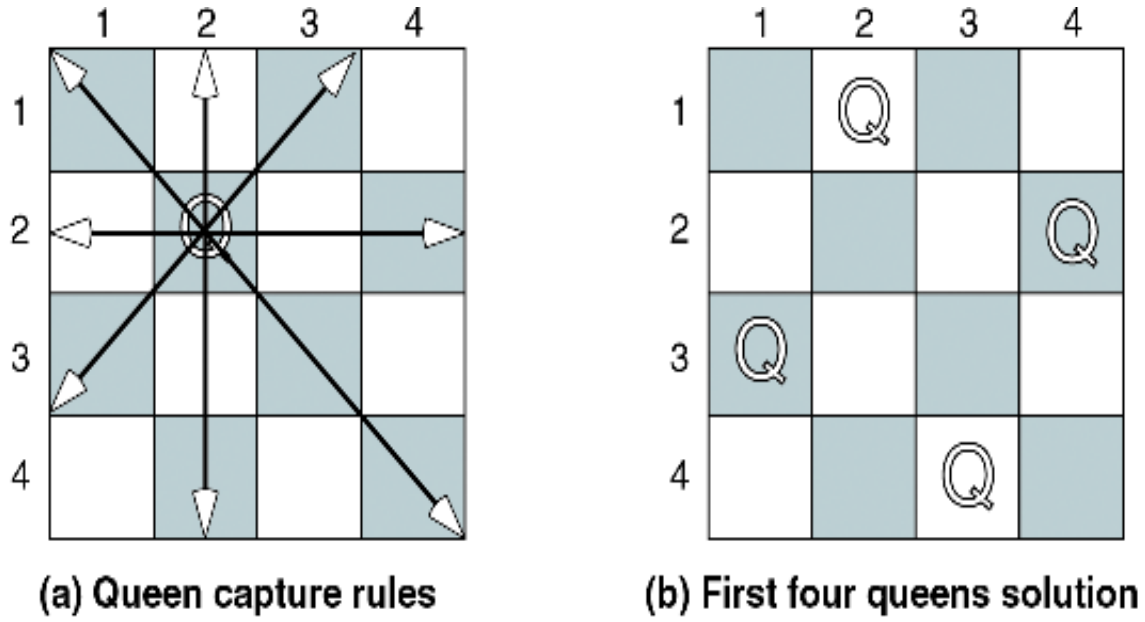


ALGORITHM 3-12 Print Path to Goal

```
Algorithm seekGoal (map)
This algorithm determines the path to a desired goal.
   Pre  a graph containing the path
   Post path printed
 1 createStack (stack)
 2 set pMap to starting point
 3 loop (pMap not null AND goalNotFound)
   1   if (pMap is goal)
       1   set goalNotFound to false
   2   else
       1   pushStack (stack, pMap)
       2   if (pMap is a branch point)
           1   loop (more branch points)
               1   create branchPoint node
               2   pushStack (stack, branchPoint)
           2   end loop
       3   end if
       4   advance to next node
   3   end if
 4 end loop
 5 if (emptyStack (stack))
   1   print (There is no path to your goal)
 6 else
   1   print (The path to your goal is:)
   2   loop (not emptyStack (stack))
       1   popStack (stack, pMap)
       2   if (pMap not branchPoint)
           1   print(map point)
       3   end if
   3   end loop
   4   print (End of Path)
 7 end if
 8 return
end seekGoal
```

Eight queens problem

Place 8 queens on the chessboard in such a way that no queen can capture another queen



(a) Queen capture rules

(b) First four queens solution

FIGURE 3-19 Four Queens Solution

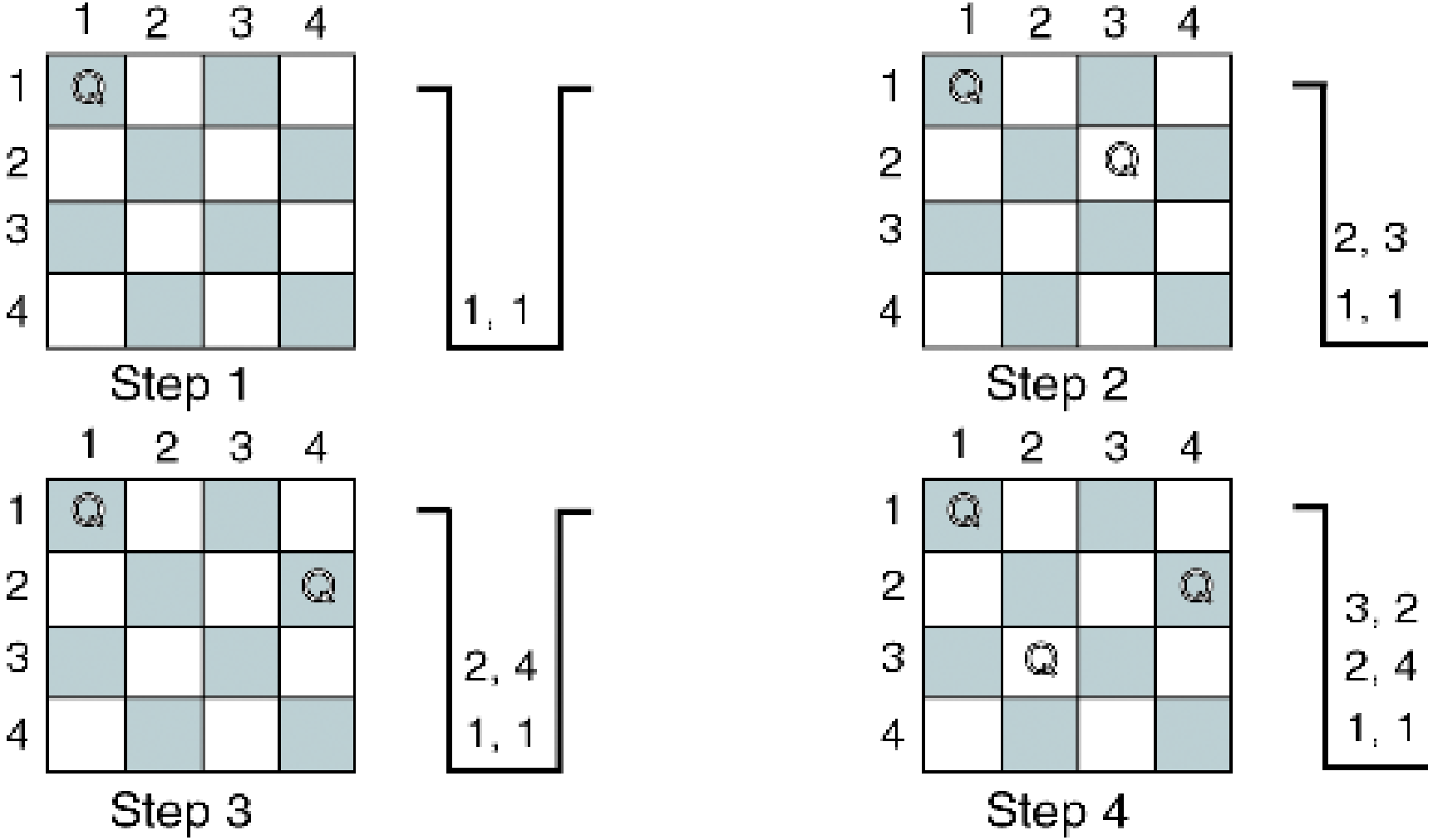


FIGURE 3-20 Four Queens Step-by-Step Solution

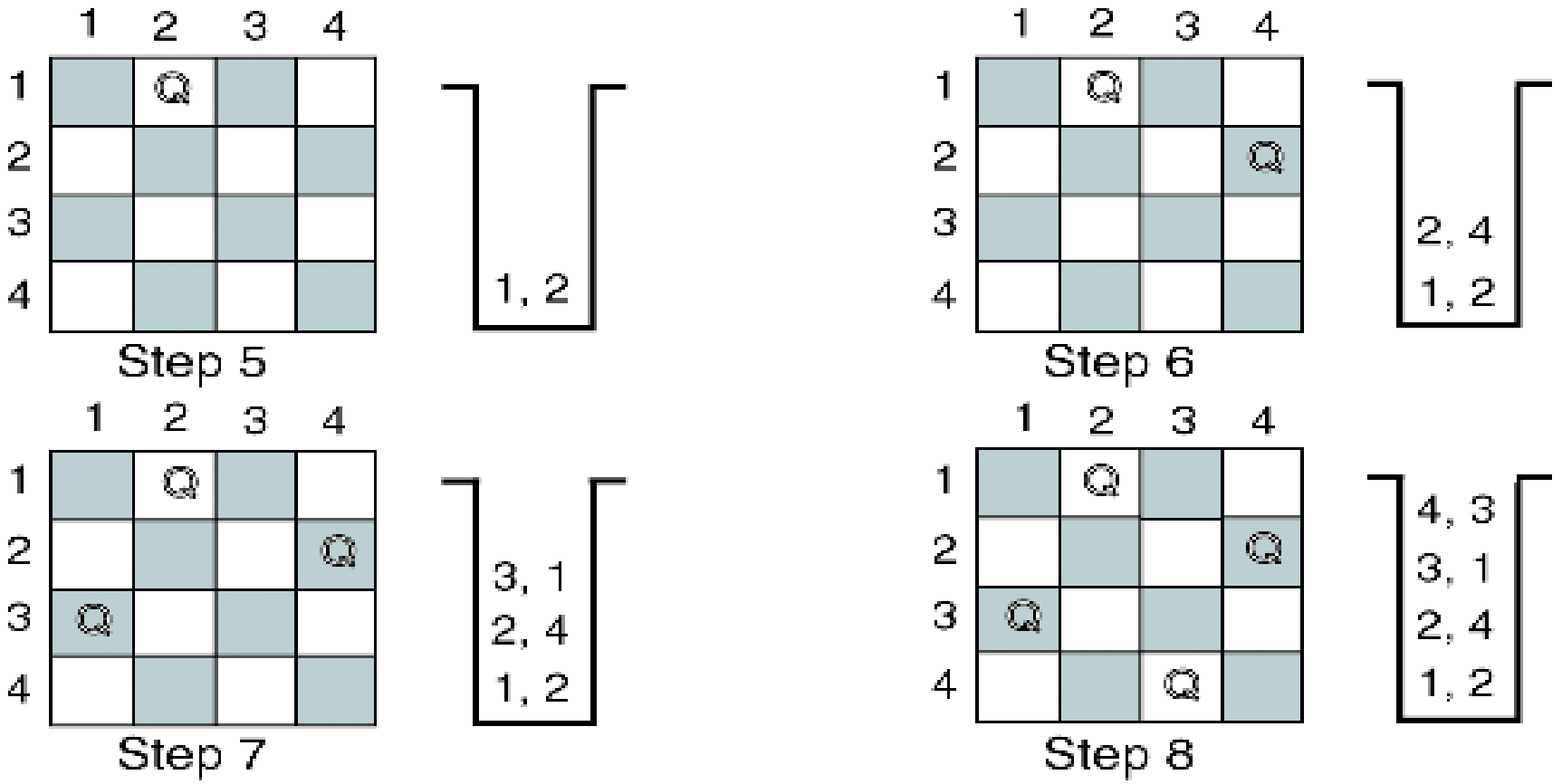


FIGURE 3-20 Four Queens Step-by-Step Solution (continued)

ALGORITHM 3-13 Eight Queens Problem

```
Algorithm queens8 (boardSize)
Position chess queens on a game board so that no queen can
capture any other queen.
   Pre  boardSize is number of rows & columns on board
   Post queens' positions printed
 1 createStack (stack)
 2 set row to 1
 3 set col to 0
 4 loop (row <= boardSize)
   1   loop (col <= boardSize AND row <= boardSize)
       1   increment col
       2   if (not guarded (row, col))
           1   place queen at row-col intersection on board
           2   pushStack (row-col into stack)
           3   increment row
           4   set col to 0
       3   end if
           At end of row. Back up to previous position.
       4   loop (col >= boardSize)
           1   popStack (row-col from stack)
           2   remove queen from row-col intersection on board
       5   end loop
   2   end loop
 5 end loop
 6 printBoard (stack)
 7 return
end queens8
```